

Programme d'Ingénierie en Intelligence Artificielle – OC IA P8 - Traitez les images pour le système embarqué d'une voiture autonome

Objectif principal

Développement d'un modèle de segmentation d'images pour systèmes embarqués de véhicules autonomes.

Contexte :

- **Mission de Future Vision Transport** : Concevoir des systèmes embarqués de vision par ordinateur pour les véhicules autonomes.
- **Objectifs spécifiques** :
 1. **Modélisation** : Entraîner et comparer des modèles de segmentation d'images pour des tâches critiques (routes, piétons, véhicules).
 2. **Industrialisation** : Concevoir une **API de prédiction** pour intégrer le modèle dans un système global.
 3. **Accessibilité** : Développer une **application web interactive** (Streamlit) pour visualiser les résultats, déployée sur le Cloud (Azure).

Approche suivie :

1. **Phase de Recherche** : Analyse comparative des architectures (**UNet Mini, VGG16 UNet, DilatedNet**).
2. **Phase Technique** : Déploiement d'une chaîne complète :
 - **API FastAPI** déployée sur Azure via Docker.
 - **Application Streamlit** pour consommer l'API et afficher les résultats.
3. **Évaluation des Performances** : Utilisation de métriques (IoU, Dice) pour guider les décisions et sélectionner le modèle optimal.

Livrables :

- **API prédictive (FastAPI)** :
URL : <http://fastapi-p8.westeurope.azurecontainer.io:8000/docs>
- **Application Streamlit** :
URL : <http://streamlit-p8.westeurope.azurecontainer.io:8501>
- **Code source** :
GitHub : https://github.com/preudh/OC_IA_P8_VoitureAutonome

Préambule

Cette présentation s'adresse à la fois aux décideurs techniques impliqués dans le développement de systèmes pour véhicules autonomes et aux spécialistes en Data Science. Elle résume l'ensemble de la démarche et les résultats d'analyse les plus pertinents.

Un compromis a été fait pour offrir des explications claires et accessibles, tout en fournissant suffisamment de détails techniques sur les étapes clés, notamment le prétraitement des données, la segmentation supervisée et le déploiement.

Bien que cette présentation n'inclue pas de code, les techniciens intéressés peuvent consulter le dépôt GitHub pour des détails approfondis et des mises en œuvre concrètes.

Les non-experts en Data Science sont encouragés à poser des questions lors de la présentation pour clarifier les concepts techniques abordés.

Sommaire

- 1) Introduction
- 2) Flux de Travail Technique
- 3) Dataset Utilisé
- 4) Présentation synthétique des différents modèles
- 5) Métriques de Performance
- 6) Modèle Avancé retenu – VGG16 UNet
- 7) Visualisation des Prédictions
- 8) Conclusion
- 9) Glossaire

1) Introduction

Développement d'une solution complète pour la segmentation d'images, intégrée dans les systèmes embarqués de véhicules autonomes.

Objectif principal : Entraîner et comparer plusieurs modèles de segmentation d'images (**DilatedNet**, **UNet Mini**, **VGG16 UNet**) pour identifier la solution offrant les meilleures performances (IoU, Dice) et la robustesse nécessaire dans des conditions réelles.

Contexte : Ce projet s'inscrit dans le cadre des activités de **Future Vision Transport**, une entreprise spécialisée dans la vision par ordinateur appliquée aux véhicules autonomes. Le dataset utilisé, **Cityscapes**, a été mis à disposition par **OpenClassrooms**. Ce dataset est spécifiquement conçu pour la segmentation sémantique urbaine, incluant des annotations sur des classes essentielles comme les routes, les piétons et les véhicules.

2) Flux de Travail Technique

Environnement : Python, PyTorch, FastAPI, Docker, Streamlit, Azure.

Déroulement :

Entraînement du modèle :

Le modèle **VGG16 UNet** a été entraîné sur le dataset **Cityscapes** à l'aide de **PyTorch**. Les couches convolutives de VGG16, pré-entraînées sur ImageNet, sont utilisées comme encodeur, et un décodeur UNet a été ajouté et entraîné pour produire des masques segmentés.

Création de l'API de prédiction :

Une API REST a été développée avec **FastAPI**, permettant d'envoyer des images et de recevoir les masques segmentés en réponse.

Lien Azure de l'API déployée :

<http://fastapi-p8.westeurope.azurecontainer.io:8000/docs>

Visualisation interactive :

Une application **Streamlit** consomme l'API **FastAPI** pour offrir une interface utilisateur interactive. Les utilisateurs peuvent charger des images, visualiser les masques segmentés en temps réel, et télécharger les résultats.

Lien interactif :

<http://streamlit-p8.westeurope.azurecontainer.io:8501>

Déploiement :

L'API FastAPI et l'application Streamlit ont été conteneurisées avec Docker. Ces conteneurs ont été déployés sur Azure.

3) Dadaset Utilisé

Données d'entrée : `leftImg8bit`

- Les images d'entrée sont situées dans le dossier `leftImg8bit` et représentent les environnements urbains à segmenter.
- **Caractéristiques des images :**
 - Résolution : **2048 x 1024 pixels**.
 - Canaux : **3 canaux (RGB)**.
- Ces images sont réparties en trois ensembles :
 1. **Jeu d'entraînement** : 2975 images.
 2. **Jeu de validation** : 500 images.
 3. **Jeu de test** : 1525 images.

Données de sortie : `gtFine`

- Les masques de segmentation sont situés dans le dossier `gtFine`.
- Seuls les fichiers ``*_labellds.png` sont **utilisés pour l'entraînement et la validation**.
 - Chaque pixel contient une valeur correspondant à une classe de segmentation (ex. : route, piéton, voiture, trottoir).
 - Le dataset inclut **34 classes**, simplifiées ici en **8 classes principales**.

Problématiques liées au dataset de test :

- Les fichiers ``*_labellds.png` apparaissent en noir car ils contiennent uniquement des identifiants de classes. Une **palette de couleurs est nécessaire pour visualiser les différentes classes de segmentation**.
- Des **hypothèses non confirmées** sur le format des masques pourraient introduire des biais lors de l'évaluation.
- **Impact : En raison de ces limitations, l'analyse des performances est principalement basée sur le dataset de validation.**

Autres fichiers dans `gtFine` :

- ``\*\_color.png`` : Visualisation des masques avec des couleurs distinctes pour chaque classe.
- ``\*\_polygons.json`` : Métadonnées décrivant les objets annotés.
- ``\*\_instancelds.png`` : Identifiants uniques pour les instances d'objets, utiles pour la segmentation d'instances.

Conclusion :

Le dataset **Cityscapes**, mis à disposition par OpenClassrooms, constitue une base de référence pour la segmentation sémantique urbaine, mais les limitations liées au dataset de test imposent une validation renforcée sur le jeu de validation.

3) Dataset Utilisé (suite)

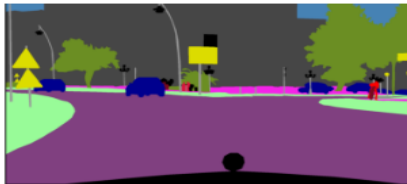
Associations dans train :

- Image aachen_000000_000019_gtFine:
- aachen_000000_000019_gtFine_color.png
- aachen_000000_000019_gtFine_instanceIds.png
- aachen_000000_000019_gtFine_labelIds.png
- aachen_000000_000019_gtFine_polygons.json

Visualisation des fichiers associés : train - <PIL.PngImagePlugin.PngImageFile image mode=L size=2048x1024 at 0x14CAC2A0E90>

aachen_000000_000019_gtFine_polygons.json

aachen_000000_000019_gtFine_color.png



aachen_000000_000019_gtFine_instanceIds.png



aachen_000000_000019_gtFine_labelIds.png



```
{
  "imgHeight": 1024,
  "imgWidth": 2048,
  "objects": [
    {
      "label": "road",
      "polygon": [
        0,
        769,
        1,
        290,
        574,
        1,
        93,
        528,
        1,
        0,
        524,
        1,
        ...
      ]
    }
  ]
}
```

Associations dans val :

- Image frankfurt_000000_000294_gtFine:
- frankfurt_000000_000294_gtFine_color.png
- frankfurt_000000_000294_gtFine_instanceIds.png
- frankfurt_000000_000294_gtFine_labelIds.png
- frankfurt_000000_000294_gtFine_polygons.json

Visualisation des fichiers associés : val - <PIL.PngImagePlugin.PngImageFile image mode=L size=2048x1024 at 0x14CAAC95A50>

frankfurt_000000_000294_gtFine_polygons.json

frankfurt_000000_000294_gtFine_color.png



frankfurt_000000_000294_gtFine_instanceIds.png



frankfurt_000000_000294_gtFine_labelIds.png



```
{
  "imgHeight": 1024,
  "imgWidth": 2048,
  "objects": [
    {
      "label": "sky",
      "polygon": [
        704,
        191,
        1,
        1044,
        404,
        1,
        1293,
        128,
        1,
        1320,
        0,
        ...
      ]
    }
  ]
}
```

3) Dataset Utilisé (suite)

Exemple de masque de segmentation et de comment fonctionne la segmentation par pixel

Dans le **masque complet**, la palette de couleurs permet de distinguer **toutes les classes** (par exemple, ciel, végétation, route).

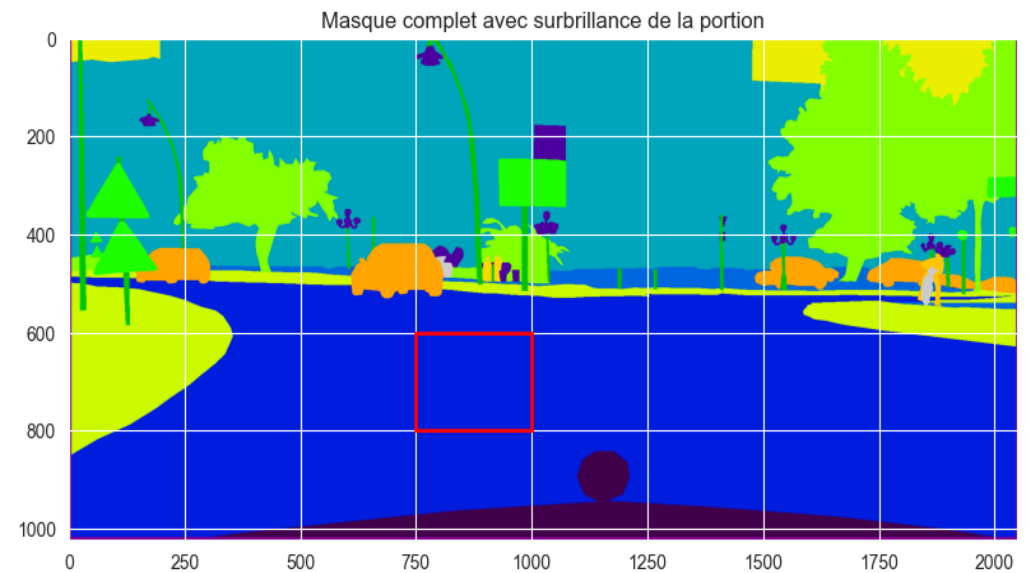
- Dans la **portion extraite**, la valeur **7** est prédominante, correspondant uniquement à la **classe "Route"**.

Le modèle doit être entraîné sur **8 catégories**. Il faut donc faire un mapping des **34 catégories** fournies par Cityscape:

Regroupement en 8 catégories principales

```
cats = {  
    'void': [0, 1, 2, 3, 4, 5, 6],  
    'flat': [7, 8, 9, 10],  
    'construction': [11, 12, 13, 14, 15, 16],  
    'object': [17, 18, 19, 20],  
    'nature': [21, 22],  
    'sky': [23],  
    'human': [24, 25],  
    'vehicle': [26, 27, 28, 29, 30, 31, 32, 33, -1]  
}
```

```
Shape du masque : (1024, 2048)  
Valeurs uniques dans le masque : [ 0  1  3  4  7  8 11 17 20 21 22 23 24 25 26 33]  
Portion du masque (200x250) :  
[[7 7 7 ... 7 7 7]  
 [7 7 7 ... 7 7 7]  
 [7 7 7 ... 7 7 7]  
 ...  
 [7 7 7 ... 7 7 7]  
 [7 7 7 ... 7 7 7]  
 [7 7 7 ... 7 7 7]]
```



4) Présentation synthétique des différents modèles

1. U-Net Mini

- **Description** : Modèle simplifié de U-Net conçu pour des environnements avec des contraintes de ressources.
- **Avantages** : Léger, rapide à entraîner, consomme peu de mémoire GPU.
- **Inconvénients** : Moins précis sur les détails fins et les classes complexes.

2. VGG16 Unet

- **Description** : Modèle basé sur VGG16 (pré-entraîné sur ImageNet) en tant qu'encodeur, avec un décodeur UNet spécialement conçu pour la segmentation.
- **Avantages** : Excellente précision grâce à des caractéristiques riches et aux skip connections.
- **Inconvénients** : Long à entraîner, gourmand en ressources GPU.

3. DilatedNet

- **Description** : Modèle utilisant des convolutions dilatées pour capturer un contexte global tout en maintenant une faible résolution spatiale.
- **Avantages** : Capacité à capturer des informations contextuelles globales.
- **Inconvénients** : Faible performance sur des tâches nécessitant des détails fins.

5) Métriques de Performance

Résumé des performances sur le dataset de validation

Modèle	Val Loss	IoU	Dice	Training Time (min)	Avantages	Inconvénients
U-Net Mini	0.462	0.588	0.697	79.14	Modèle léger, rapide à entraîner.	Légèrement moins performant que VGG16 U-Net.
VGG16 U-Net	0.412	0.607	0.713	153.60	Meilleures métriques globales.	Long à entraîner et gourmand en ressources.
DilatedNet	0.799	0.391	0.503	138.13	-	Faibles performances globales.

U-Net Mini : Ce modèle léger offre un bon compromis entre **vitesse d'entraînement** et performance, mais il est **moins précis** sur les classes détaillées. Il convient bien aux environnements avec des contraintes de calcul.

VGG16 UNet : Ce modèle fournit les **meilleures métriques globales** (IoU de 0.607 et Dice de 0.713), démontrant une bonne précision sur les classes critiques comme les routes et les piétons. Cependant, son **temps d'entraînement est long**, et il nécessite des ressources élevées.

DilatedNet : Avec un IoU de 0.391 et un Dice de 0.503, ce modèle est le moins performant sur le dataset de validation. Il a des **difficultés à capturer les détails fins** et les structures complexes, ce qui le rend moins adapté pour des tâches critiques comme la segmentation pour véhicules autonomes.

Conclusion :

Le **VGG16 UNet** est le modèle le plus adapté grâce à ses performances élevées, mais son coût en calcul est un inconvénient à considérer. **U-Net Mini** reste une option viable pour des environnements où la rapidité est essentielle, tandis que **DilatedNet** montre ses limites sur ce type de tâche.

6) Modèle Avancé Retenu – VGG16 UNet

VGG16 UNet : Architecture et Entraînement Transfert Learning avec VGG16 :

Les couches convolutives de l'**encodeur VGG16** pré-entraînées sur **ImageNet** sont utilisées pour extraire des caractéristiques hiérarchiques des images.

Ces couches sont réutilisées **telles quelles** (gelées) et **ne sont pas réentraînées**, ce qui réduit le temps de calcul tout en exploitant des connaissances générales sur les images.

Décodeur UNet :

Les couches du décodeur, conçues spécifiquement pour la segmentation, **sont entièrement entraînées à partir de zéro**. Elles **transforment les caractéristiques extraites par VGG16 en masques segmentés**.

Les skip connections relient les couches du décodeur à celles de l'encodeur, permettant de récupérer les détails perdus lors des étapes de compression.

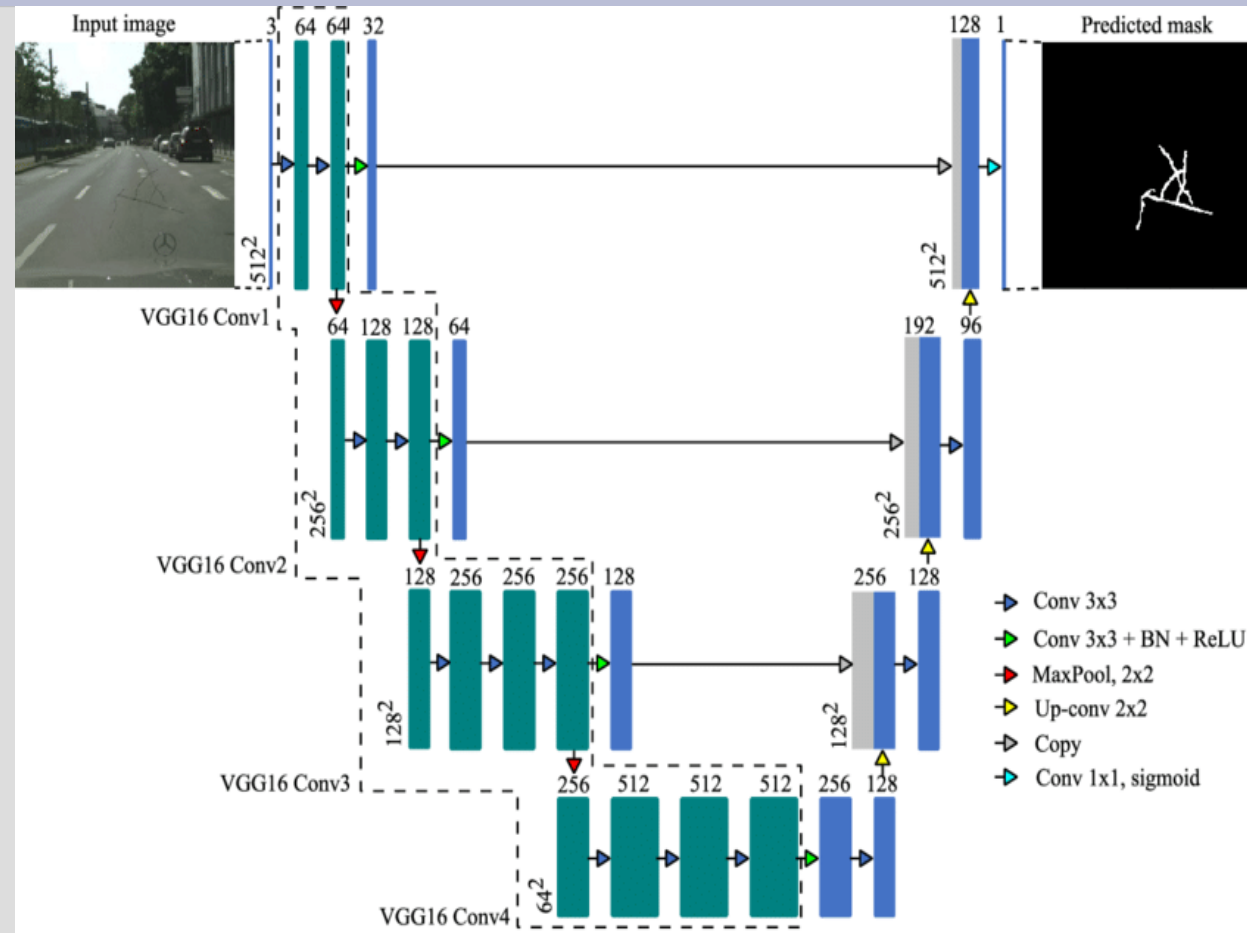
La dernière couche ajuste la sortie pour segmenter les pixels en fonction des **8 classes principales**.

Entraînement supervisé :

L'entraînement se fait uniquement sur les couches du **décodeur**. Les gradients ne sont pas calculés pour les couches gelées de l'encodeur.

La **perte utilisée est la CrossEntropyLoss**, adaptée à une tâche de segmentation multi-classes.

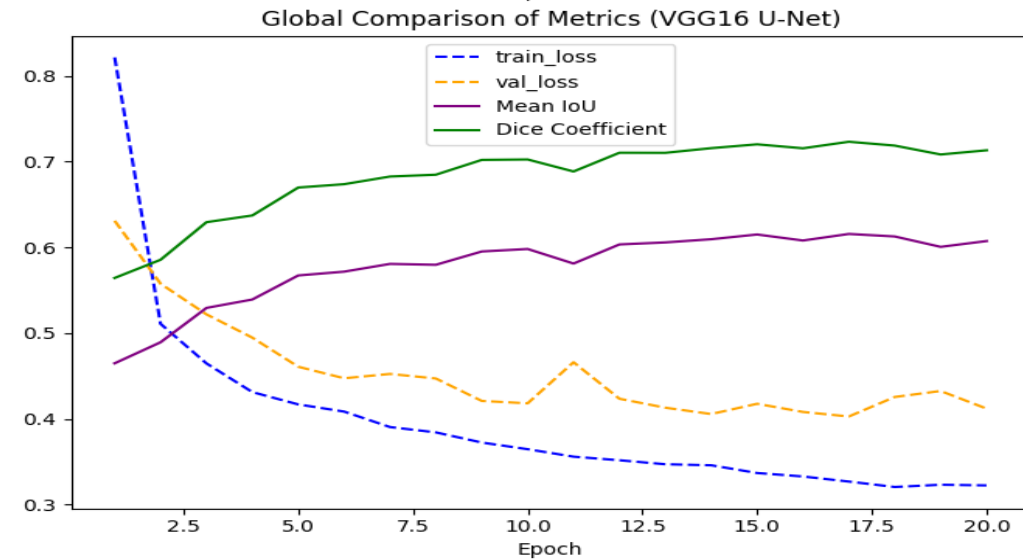
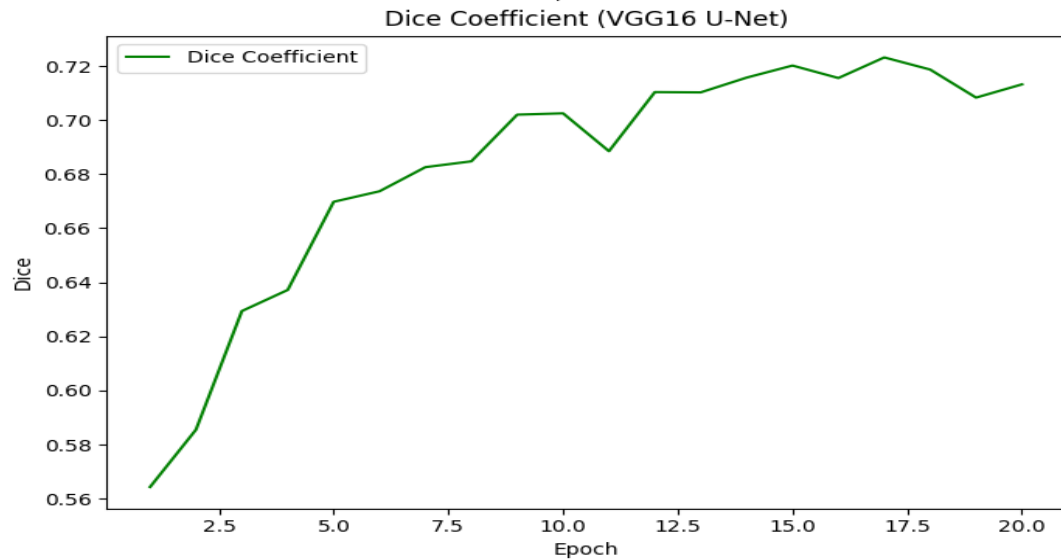
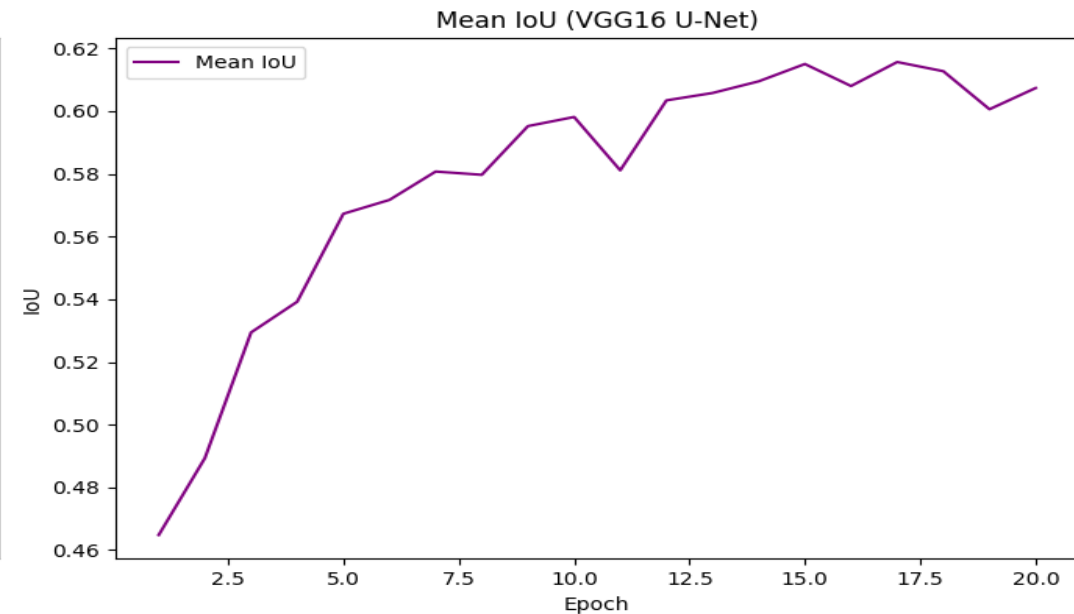
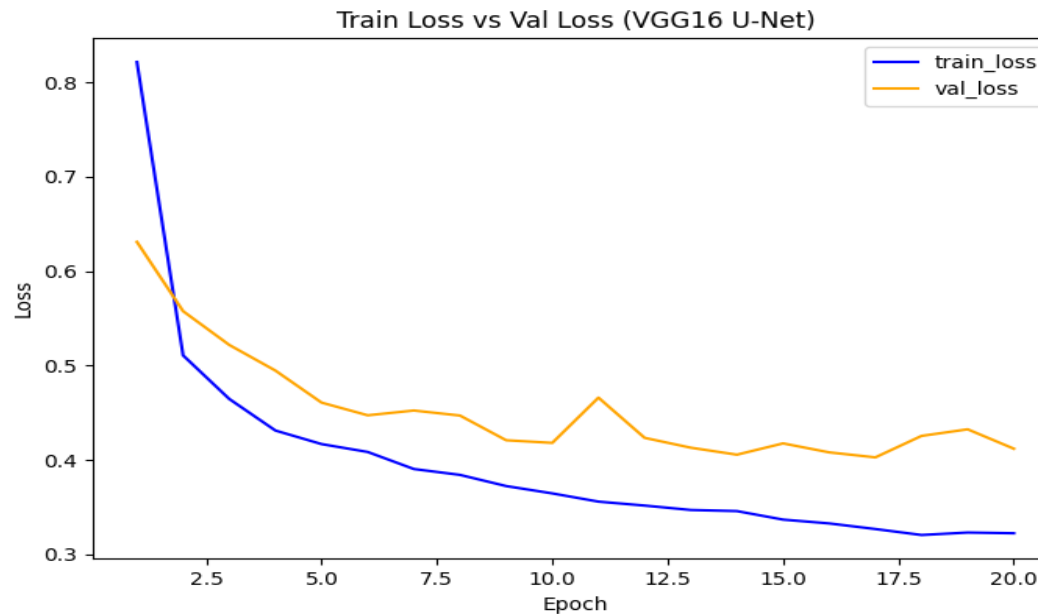
Le **modèle est optimisé avec Adam**, avec un apprentissage réglé pour converger en **25 époques maximum**, avec un arrêt anticipé après **3 époques sans amélioration de la perte de validation**.



Avantages : Réduction du temps d'entraînement grâce aux couches pré-entraînées de VGG16, haute précision grâce aux skip connections et au décodeur, et flexibilité pour d'autres tâches.

Inconvénients : Consommation élevée en mémoire GPU et limites des connaissances pré-apprises sur ImageNet pour des environnements urbains.

6) Modèle Avancé Retenu – VGG16 Unet (suite)



7) Visualisation des Prédictions

Outil : Dashboard Streamlit

Streamlit est une bibliothèque Python open-source qui permet de concevoir facilement des applications web interactives. Elle ne nécessite pas de compétences avancées en développement web et est idéale pour les projets d'intelligence artificielle.

URL du tableau de bord :

<http://streamlit-p8.westeurope.azurecontainer.io:8501>

Fonctionnalités

Chargement d'images : L'utilisateur peut sélectionner une image à segmenter via l'interface.

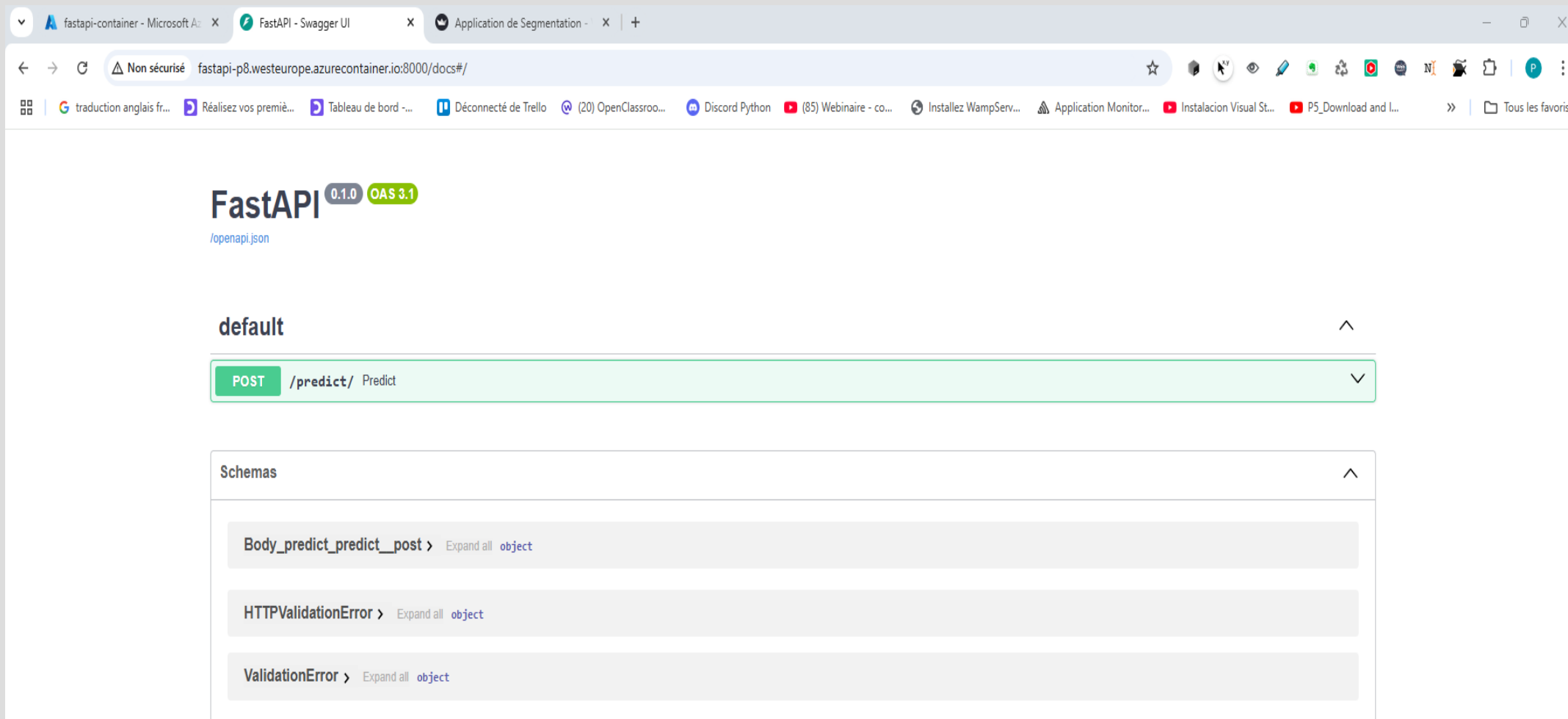
Prédictions en temps réel : Le tableau de bord affiche le masque segmenté pour l'image sélectionnée, en utilisant le modèle VGG16 UNet.

Interface accessible : Permet une utilisation intuitive, adaptée aux non-techniciens, avec des explications claires sur les résultats affichés.

Ce tableau de bord a été conçu pour simplifier l'analyse des prédictions et offrir une visualisation des masques segmentés de manière interactive.

7) Visualisation des Prédictions (suite)

L'interface **Swagger de FastAPI** permet de tester l'API directement via une documentation interactive. L'utilisateur peut envoyer une image au point de terminaison `/predict` pour recevoir le masque segmenté correspondant, simplifiant ainsi les tests et l'intégration.



7) Visualisation des Prédictions (suite)

Cette interface **Streamlit** permet de télécharger une image et de visualiser immédiatement le masque segmenté généré par le modèle.

fastapi-container - Microsoft A... x FastAPI - Swagger UI x Application de Segmentation - x +

Non sécurisé streamlit-p8.westeurope.azurecontainer.io:8501

traduction anglais fr... Réalisez vos premiè... Tableau de bord ~... Déconnecté de Trello (20) OpenClassroo... Discord Python (85) Webinaire - co... Installez WampServ... Application Monitor... Instalacion Visual St... P5_Download and I... Tous les favoris

🚗 Application de Segmentation pour Véhicules Autonomes

Cette application utilise des modèles d'intelligence artificielle pour segmenter des images en 8 catégories principales :

- **void** : Éléments d'arrière-plan (0, 1, 2, 3, 4, 5, 6)
- **flat** : Routes, trottoirs, etc. (7, 8, 9, 10)
- **construction** : Bâtiments, clôtures, murs (11, 12, 13, 14, 15, 16)
- **object** : Poteaux, panneaux de signalisation, feux de circulation (17, 18, 19, 20)
- **nature** : Végétation et terrain (21, 22)
- **sky** : Ciel (23)
- **human** : Piétons, cyclistes (24, 25)
- **vehicle** : Voitures, camions, bus, motos (26 à 33, -1)

Déposez votre image ici ou cliquez pour télécharger

Drag and drop file here
Limit 200MB per file • JPG, PNG, JPEG

Browse files

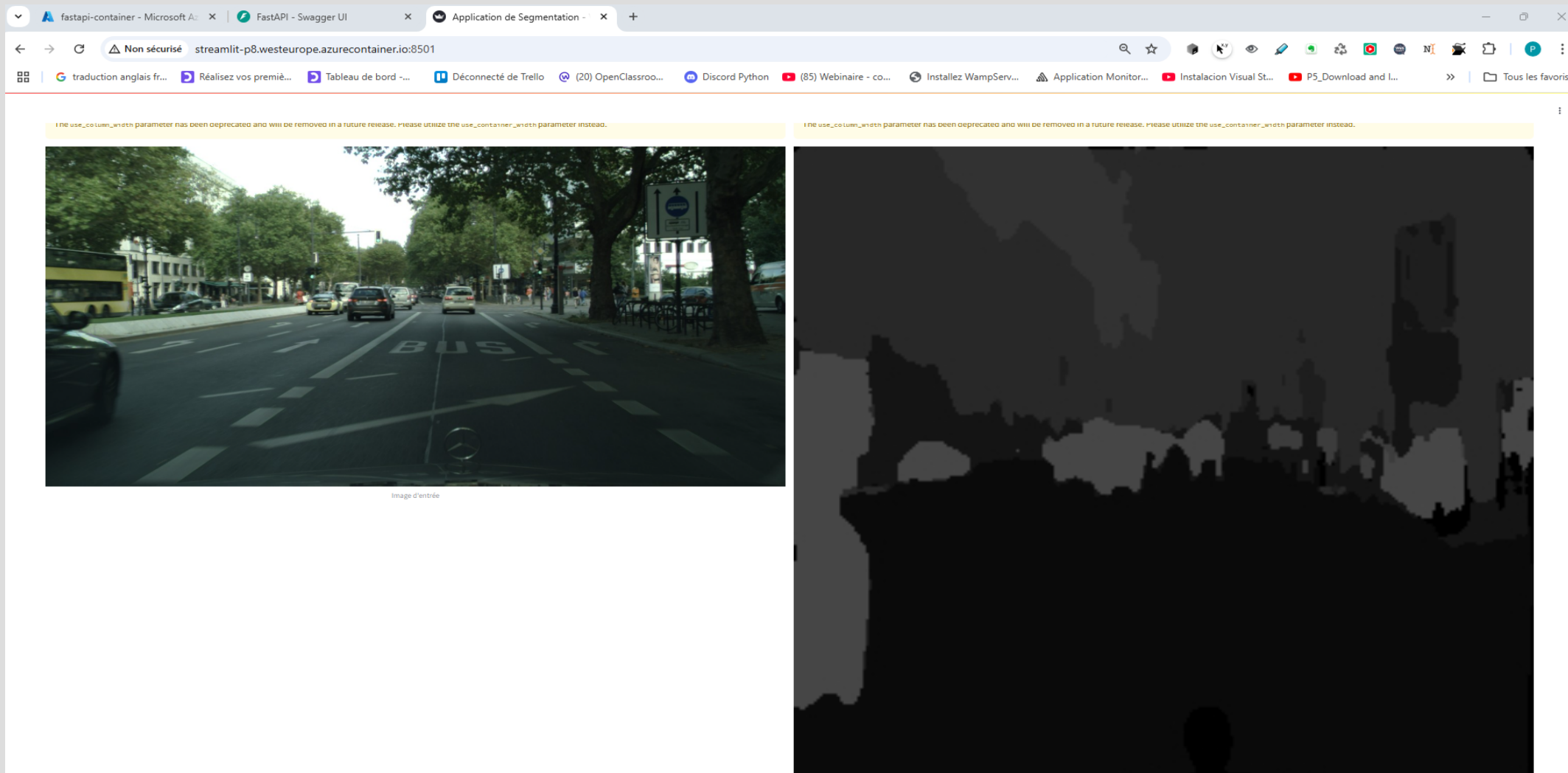
berlin_000000_000019_leftImg8bit.png 2.2MB

The use_column_width parameter has been deprecated and will be removed in a future release. Please utilize the use_container_width parameter instead.



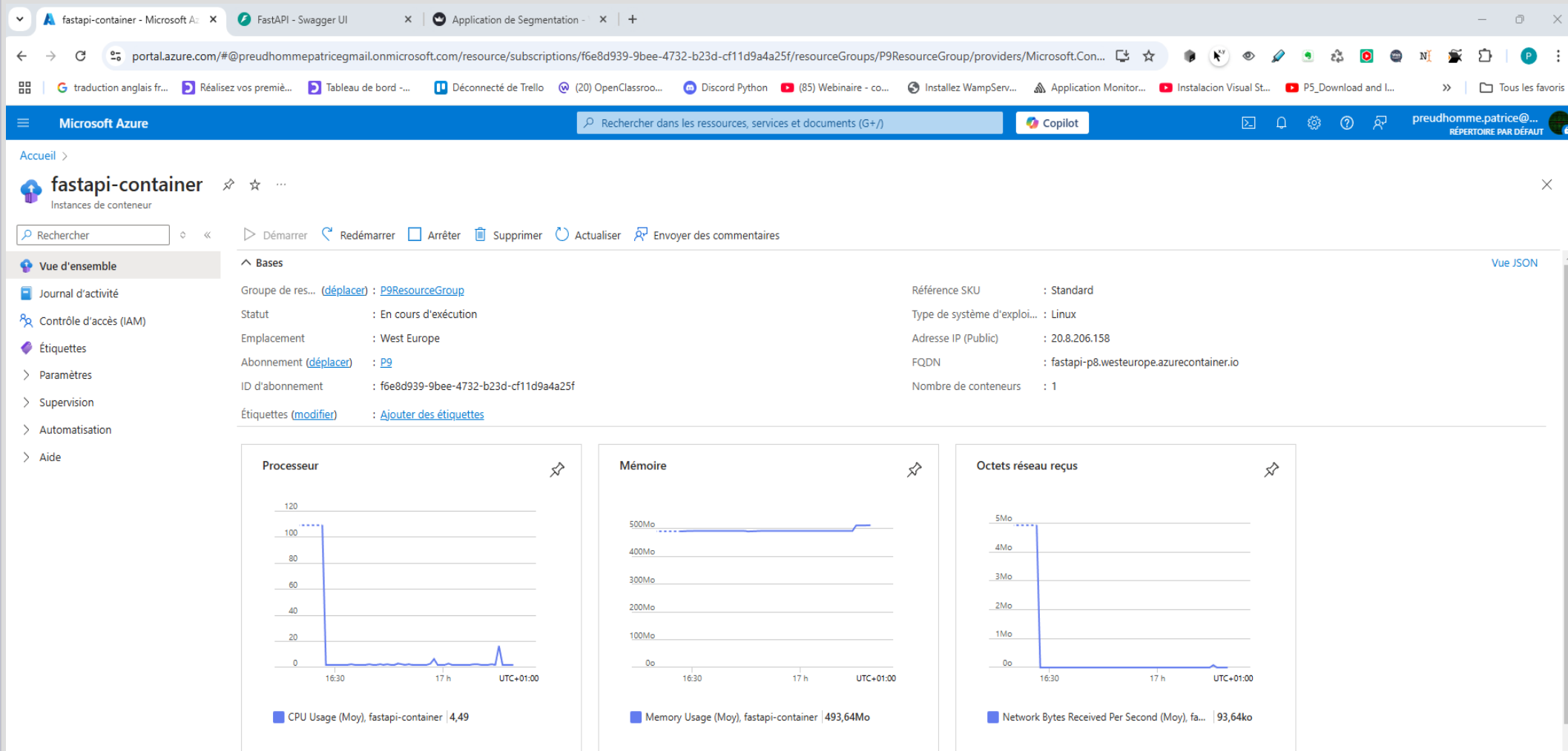
7) Visualisation des Prédictions (suite)

Lors de la soumission de l'image via Streamlit, celle-ci est envoyée à l'API FastAPI via une requête POST au point de terminaison /predict. L'API traite l'image, génère le masque segmenté via le modèle VGG16 UNet, puis renvoie ce masque en nuances de gris à l'application pour affichage.



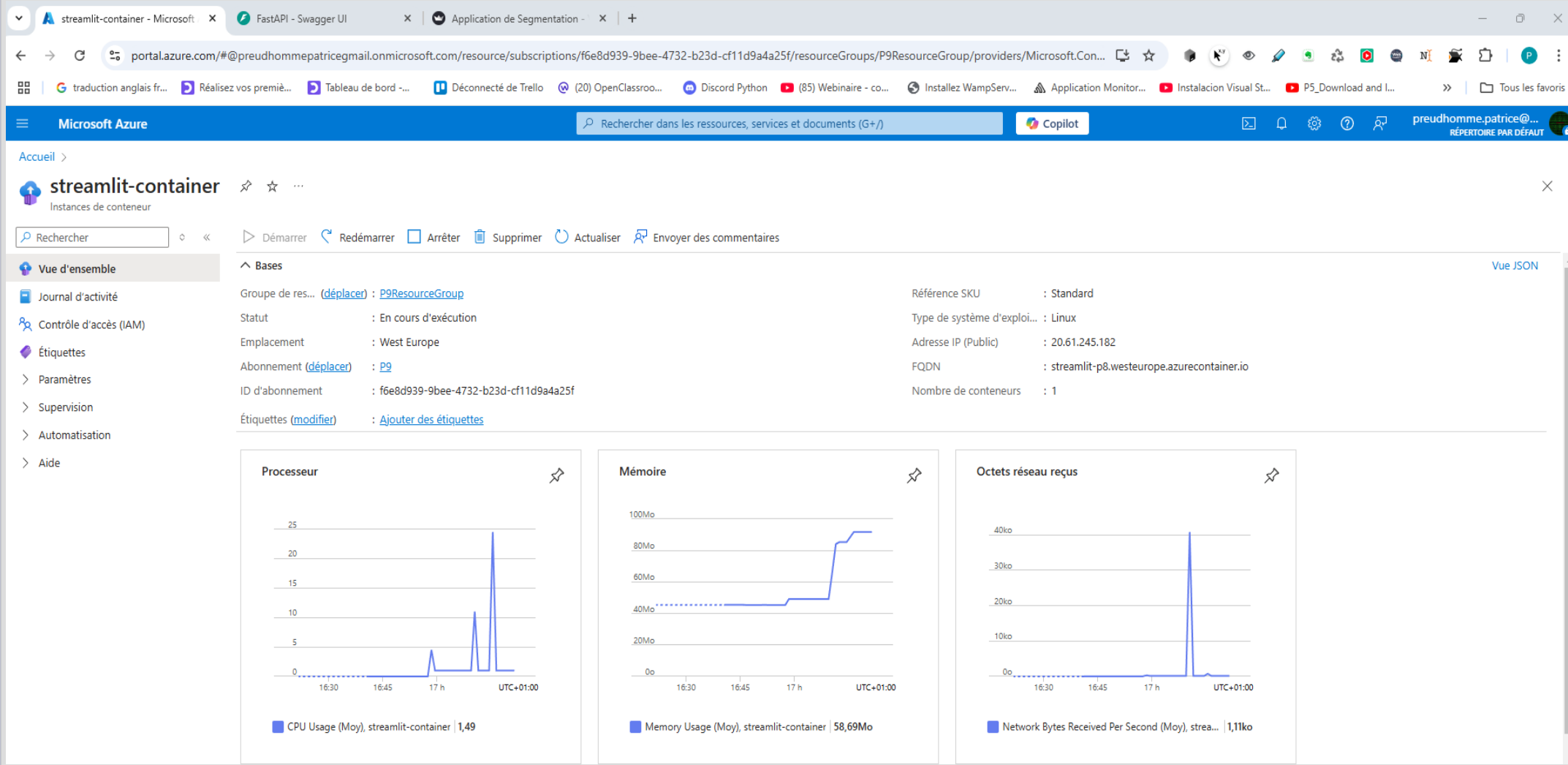
7) Visualisation des Prédictions (suite)

Déploiement : sur Azure en utilisant Docker qui repose sur l'utilisation de conteneurs, qui isolent les applications et leurs dépendances dans des environnements légers, portables et reproductibles, permettant de les exécuter de manière identique entre une machine locale et un serveur cloud, comme Azure.



7) Visualisation des Prédictions (suite)

Déploiement : sur Azure en utilisant Docker qui repose sur l'utilisation de conteneurs, qui isolent les applications et leurs dépendances dans des environnements légers, portables et reproductibles, permettant de les exécuter de manière identique entre une machine locale et un serveur cloud, comme Azure.



8) Conclusion

Synthèse des résultats

- Le **VGG16 UNet** a démontré les meilleures performances en termes de précision pour la segmentation, bien que son coût en ressources GPU soit élevé.
- Les modèles **U-Net Mini** et **DilatedNet** présentent des avantages pour des scénarios spécifiques, mais leurs performances globales sont inférieures.

Points critiques

- **Limitation liée au dataset de test :**

Le **dataset de test** utilise un format particulier où **les masques apparaissent noirs**, chaque pixel portant un entier correspondant à une classe.

Ce format nécessite potentiellement **un mapping vers des couleurs pour rendre les classes visibles et exploitables**. Il n'est pas garanti que ce format sera utilisé pour les données futures.

Sans collaboration avec l'équipe ayant généré ces données (Franck), il est impossible de confirmer si les conversions effectuées sont correctes ou si ce format est représentatif des futurs datasets.

Cette incertitude constitue un blocage décisionnel majeur, ce qui maintient le projet dans le cadre d'un MVP (Minimum Viable Product) sans possibilité d'industrialisation immédiate.

8) Conclusion (suite)

Pistes de développement supplémentaires

Automatisation avec CI/CD :

- Mettre en place un pipeline CI/CD avec GitHub Actions pour :

- *Tester automatiquement les changements.
- *Construire les images Docker.
- *Déployer les conteneurs via Azure Container Registry (ACR).

Exploration d'autres modèles :

- *DeepLabV3 : Modèle avancé pour la segmentation sémantique, utilisant des convolutions atrous pour améliorer la capture de contexte global.
- *Autres architectures modernes basées sur les Transformers ou l'attention pour capturer des relations complexes dans les images.

9) Glossaire

1. IoU (Intersection over Union)

Définition : Mesure de la précision de la segmentation en comparant l'intersection entre le masque prédit et le masque réel, par rapport à leur union.

Formule : $\text{IoU} = \text{Aire de l'intersection} \div \text{Aire de l'union}$

Interprétation : Un IoU élevé (> 0.7) indique une bonne correspondance entre la prédiction et la vérité terrain.

2. Dice Coefficient

Définition : Mesure de la similarité entre deux ensembles, utilisée pour évaluer la segmentation.

Formule : $\text{Dice} = 2 \times \text{Aire de l'intersection} \div (\text{Aire totale des deux ensembles})$

Interprétation : Similaire à l'IoU mais plus sensible aux petites régions. Un Dice élevé (> 0.85) indique une excellente précision.

3. Transfert Learning

Définition : Technique où un modèle pré-entraîné sur un large dataset (ex. : ImageNet) est réutilisé pour une tâche spécifique.

Avantages : Réduction du temps d'entraînement et amélioration des performances grâce aux connaissances déjà acquises.

4. FastAPI

Définition : Framework web en Python permettant de créer des API REST rapides, simples, et performantes.

Utilisation dans le projet : FastAPI permet de créer une API REST qui reçoit des images via des requêtes HTTP POST (point de terminaison /predict), applique le modèle VGG16 UNet, et retourne le masque segmenté correspondant.

5. Docker

Définition : Plateforme permettant de conteneuriser des applications pour garantir leur portabilité et leur cohérence.

Utilisation dans le projet : Déploiement de l'API FastAPI et de l'application Streamlit sur Azure.